Build a Mini-Game with AI, Host It on Vercel, and Prepare It for Monetization

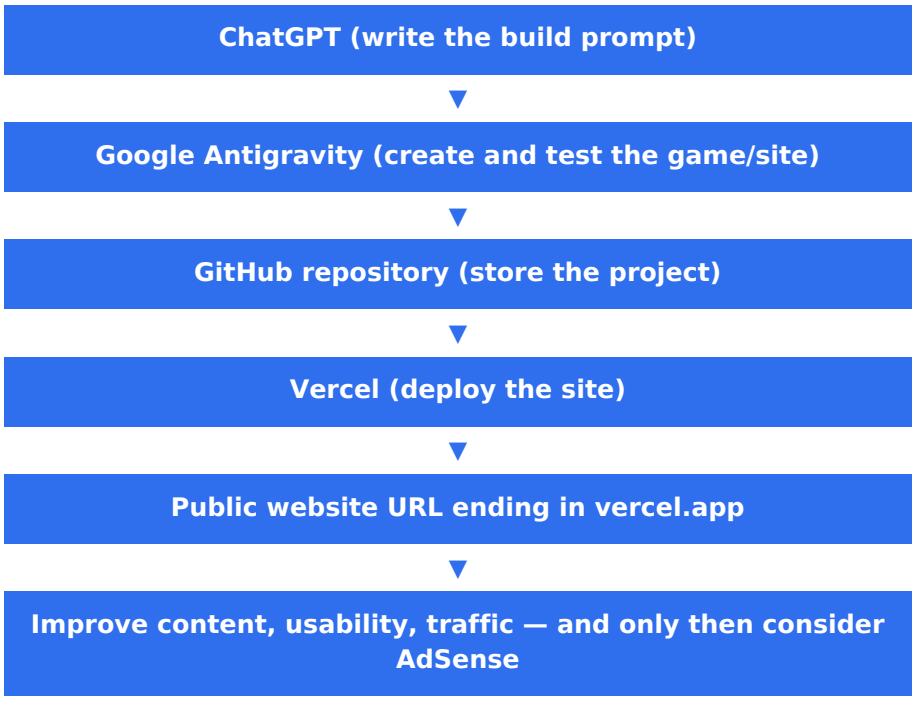
Beginner-friendly walkthrough | Updated: October 2026

What this guide helps you do

1. Generate a browser mini-game using ChatGPT.
2. Build and test it in Google Antigravity.
3. Create an arcade website with multiple games.
4. Publish the website online using Vercel's free tier, subject to its current plan limits and terms.
5. Understand what is needed before applying for Google AdSense.

> **Important:** Hosting a game does not automatically generate traffic or income. AdSense approval is not guaranteed. Treat revenue as a possibility, not a promise.

Part 1 — The Simple Workflow



Useful official resources

- Google Antigravity docs: https://www.antigravity.google/docs/home
- Vercel + Vite guide: https://vercel.com/docs/frameworks/frontend/vite
- Deploy a Git repository on Vercel: https://vercel.com/docs/git
- Vercel deployment overview: https://vercel.com/docs/deployments/overview
- Google AdSense site approval help: https://support.google.com/adsense/answer/81904?hl=en

Part 2 — Step 1: Generate a Good Game Prompt in ChatGPT

Open ChatGPT and paste the following prompt. Change the game concept if you want to build a different game.

COPY-PASTE PROMPT

PROMPT 1 — BUILD A COMPLETE MINI-GAME

Act as a senior HTML5 game developer and front-end engineer.

Build a complete, polished, browser-based fruit-slicing mini-game that runs on desktop and mobile.

TECH STACK:

- Prefer plain HTML, CSS, and JavaScript for a lightweight static site.
- If a framework is genuinely useful, use React + Vite.
- Do not use a backend, database, paid API, or API key.
- The game must be deployable as a static website on Vercel.
- Do not rely on files or APIs that exist only on your local computer.

GAMEPLAY:

- Show fruit objects moving across the play area.
- Let players slice fruit using mouse movement or touch gestures.
- Include score, timer, start, pause, restart, and game-over states.
- Add simple particle effects when fruit is sliced.
- Add a combo bonus for consecutive successful slices.
- Add a clear “avoid bombs” rule if bombs are included.
- Make controls responsive and touch-friendly.
- Prevent accidental page scrolling while the player is actively swiping inside the game area, without breaking normal page navigation elsewhere.
- Provide a clear first-time instruction overlay.
- Include a sound on/off toggle.
- Use generated shapes, CSS, or original simple vector graphics rather than copyrighted assets copied from other games.
- If sound assets are unavailable, create simple optional effects with Web Audio only after a user interaction, and provide a mute option.

DESIGN:

- Premium, clean, bright arcade aesthetic.
- Responsive layout for phone, tablet, and desktop.
- Smooth animations, readable typography, attractive score display, and polished buttons.
- Do not copy a famous game’s exact branding, artwork, or interface.
- Avoid intrusive pop-ups and fake download buttons.

QUALITY:

- Make the game fully playable, not just a mockup.
- Handle start, pause, restart, game over, and score correctly.
- Check for console errors.
- Test keyboard, mouse, and touch controls.
- Ensure the layout fits small screens.

- Do not leave TODO placeholders or non-working buttons.
- Include all necessary source files.
- Include a README with local testing and deployment instructions.

DELIVERABLE:

Create the complete project in the current workspace. Before finishing, run the available build/test commands, fix errors, and tell me exactly how to run it locally and deploy it to Vercel.

Do not merely explain or output a code snippet. Create the actual files in the workspace if your environment supports file editing.

Part 3 — Step 2: Build It in Google Antigravity

1. Install and open Google Antigravity from its official website/docs.
2. Create a new project/workspace folder, for example: mini-game-arcade.
3. Open that folder in Antigravity.
4. Paste PROMPT 1 into the agent/chat panel.
5. Allow the agent to create the files and inspect its changes.
6. Ask it to run the project and test it in a browser.
7. Test the game yourself (see the questions below).
8. If the project uses Vite/React, ask the agent to run: npm install then npm run dev. Open the local URL it provides, commonly http://localhost:5173.
9. For a production check, ask the agent to run: npm run build
10. Do not proceed to deployment until the build succeeds and the game works.

When testing the game yourself, ask:

- Can you start and restart it?
- Does scoring work?
- Do mouse and touch controls work?
- Does it fit a phone screen?
- Are there console errors?

If Antigravity cannot run a command or open a browser in your environment, ask it to explain the limitation and give you the exact manual step. Do not assume every environment has the same permissions.

COPY-PASTE PROMPT

PROMPT 2 — TEST AND FIX THE GAME

Audit the current mini-game project as a senior QA engineer.

1. Run the available development server and production build.
2. Test start, pause, restart, timer, score, game-over, and sound toggle.
3. Test mouse and touch input.
4. Check responsiveness at narrow mobile width and desktop width.
5. Check browser console errors and missing assets.
6. Check that the project uses relative/local assets correctly and does not depend on files outside the project folder.
7. Fix actual bugs, not just describe them.

8. Do not remove working features or redesign everything unnecessarily.
9. Run the production build again after fixes.
10. Report what you tested and any limitation that remains.

Part 4 — Step 3: Turn One Game into an Arcade Website

Once the first game works, you can add more games. Build one game properly first; then expand. Ten unfinished games are not better than one good game.

COPY-PASTE PROMPT

PROMPT 3 — BUILD A MINI-GAME ARCADE WEBSITE

Act as a senior front-end developer and product designer.

Using the existing working mini-game project, create a responsive arcade website called “Mini Game Arcade”.

REQUIREMENTS:

- Keep the existing game working.
- Create a clean homepage with a short, original introduction explaining what the site offers.
- Display game cards in a responsive grid.
- Each card must have a title, original thumbnail/illustration, short description, and Play button.
- Create a dedicated playable page or route for each completed game.
- Start with the existing fruit-slicing game.
- If other games are not implemented yet, show only the games that actually work. Do not create fake cards that lead nowhere.
- Make it easy to add more games later.
- Add search or category filters only if they work correctly.
- Include About, Contact, Privacy Policy, and Terms pages/sections with clear navigation.
- Include a simple explanation of controls for each game.
- Make the site mobile-friendly, fast, accessible, and keyboard usable.
- Use original artwork or assets that I have permission to use.
- Do not copy copyrighted game art, logos, or code.
- Do not add fake testimonials, fake user counts, fake earnings, or fake reviews.
- Do not add ad placeholders that block gameplay.
- Do not require an API key or paid backend.
- Keep the project deployable as a static site on Vercel.
- Include a README that explains how to add a new game.
- Test all links and buttons and run the production build.

Create the actual files in the current workspace, inspect the result, fix errors, and summarize the changes.

Part 5 — Step 4: Prepare the Project for Vercel

First identify which kind of project Antigravity created.

A) Simple static HTML/CSS/JS

The project may contain index.html, styles.css, game.js, and asset files. There may be no package.json. A static site can often be deployed directly.

B) Vite / React

The project usually contains package.json, src/, public/, and a Vite configuration. Common settings are:

- Build command: npm run build
- Output directory: dist
- Install command: npm install (Vercel usually detects the package manager)

Do not blindly use these settings if the project's package.json or framework says otherwise.

COPY-PASTE PROMPT

PROMPT 4 — DEPLOYMENT READINESS CHECK

Inspect this project specifically for Vercel deployment.

1. Identify the framework and the correct project root.
2. Inspect package.json and identify the real build command.
3. Run the production build and fix errors.
4. Identify the correct output directory.
5. Check for missing assets, broken routes, case-sensitive filename problems, and hard-coded localhost URLs.
6. Confirm that no secret API keys or passwords are committed.
7. Confirm that all required game assets are inside the project or hosted at legitimate public URLs.
8. If client-side routes are used, determine whether a Vercel rewrite is required and add only the correct configuration.
9. Do not add a vercel.json file unless the project actually needs it.
10. Write a short DEPLOYMENT.md with exact settings for Vercel.
11. Do not claim deployment succeeded unless the site was actually deployed and tested.

Part 6 — Step 5: Upload the Project to GitHub

The GitHub method is recommended for beginners because Vercel can automatically redeploy when you push updates.

1. Create or sign in to a GitHub account: https://github.com/
2. Create a new repository. Suggested name: mini-game-arcade. For a public portfolio/demo, a public repository is convenient. A private repository can also work if your Vercel account has access.
3. In Antigravity, ask the agent to prepare the project for GitHub (use Prompt 5 below).

COPY-PASTE PROMPT

PROMPT 5 — PREPARE FOR GITHUB

Prepare this project for a clean GitHub repository.

– Inspect the files before changing anything.
– Create or verify a suitable .gitignore for the actual stack.

- Exclude node_modules, build output if appropriate, local environment files, credentials, and operating-system clutter.
- Never put API keys, passwords, tokens, or secrets in source code.
- Keep all files needed to build and run the project.
- Provide the exact Git commands to initialize the repository, commit the project, and push it to the GitHub repository I specify.
- Do not run destructive commands or overwrite an existing repository.
- If Git is already initialized, inspect its current status and remote before suggesting commands.

4. Follow the commands the agent provides. A typical new-repository sequence may resemble the following, but replace the URL with your own repository URL and follow GitHub's current instructions:

```
git init
git add .
git commit -m "Initial mini-game arcade"
git branch -M main
git remote add origin YOUR_GITHUB_REPOSITORY_URL
git push -u origin main
```

Do not blindly rerun git init or add a remote if the project already has Git configured. If authentication or push errors appear, resolve them rather than deleting project files.

Part 7 — Step 6: Deploy on Vercel Using the Dashboard

Official Vercel dashboard: https://vercel.com/

1. Create/sign in to your Vercel account.
2. Choose "Add New…" or "New Project" (wording may vary).
3. Connect your GitHub account if prompted.
4. Import the mini-game-arcade repository.
5. Confirm the project root directory. It should be the folder containing the app's package.json or index.html, depending on the project.
6. Check the framework preset. Vercel usually detects common frameworks automatically.
7. For a typical Vite app, confirm the build command is npm run build and output directory is dist. Check the actual project before accepting these values.
8. For a plain static HTML site, configure the root/output settings according to the actual folder structure; do not invent a build command if none is required.
9. Add environment variables only if the project genuinely needs them. Never paste secrets into client-side code.
10. Click Deploy.
11. Wait for the build logs to finish.
12. Open the generated URL, usually ending in .vercel.app.
13. Test the live site on desktop and mobile.
14. Test the game, images, navigation, refresh behavior, and all buttons.

15. Save the deployment URL.

Vercel's Git integration can create deployments from repository commits and automatically update deployments when changes are pushed to the connected production branch. Check the current Vercel plan limits and acceptable-use terms before publishing a high-traffic project.

Official documentation

- Vercel Git deployment: https://vercel.com/docs/git
- Vercel deployments: https://vercel.com/docs/deployments/overview
- Vite on Vercel: https://vercel.com/docs/frameworks/frontend/vite

Part 8 — Alternative: Deploy with Vercel CLI

Use this route if you prefer a terminal or your GitHub flow is not available.

Prerequisites:

- Node.js installed if the project requires it.
- A working project with a successful production build.
- A Vercel account.

From the project root, open a terminal, then run:

```
# Install the CLI
npm install --global vercel

# Log in
vercel login

# Preview deployment (follow the prompts to link/create the project)
vercel

# Production deployment
vercel --prod
```

Inspect the generated preview URL and test it before running the production deployment. Vercel CLI documentation: https://vercel.com/docs/cli/deploy

Do not run these commands from the wrong directory. The project root is the folder containing the app's entry files/package.json. If a command fails, read the actual error; do not repeatedly run random commands.

Part 9 — Step 7: Add More Games

Add games incrementally. Test each one before publishing.

COPY-PASTE PROMPT

PROMPT 6 — ADD A SECOND GAME

Add a new "Memory Match" game to the existing arcade project.

– Preserve all existing working games and routes.

- Follow the site’s current design system.
- Create a fully playable card-matching game with a timer, move counter, restart button, win state, and accessible keyboard/touch interactions.
- Use original simple illustrations or shapes.
- Make it responsive and mobile-friendly.
- Add the new game to the homepage only after it is functional.
- Do not create fake game cards or broken Play buttons.
- Test the game and run the production build.
- Explain which files were added or changed.

For each additional game, replace the game name and rules in the prompt. Possible concepts:

Bubble Pop	Memory Match	Color Reaction
Brick Breaker	Number Puzzle	Space Dodge
Tap Challenge	Block Puzzle	Quick Reflex

Avoid copying a famous game’s branding, copyrighted assets, or exact presentation. A list of ten cards is not enough; each listed game should actually work.

Part 10 — Step 8: Prepare for Google AdSense

Important: Vercel hosting does not automatically activate AdSense, and a deployed site is not guaranteed to be approved.

Before applying

1. Make sure the website is complete and publicly accessible.
2. Provide genuinely useful, original content. A site containing only game thumbnails or a bare template may not meet quality expectations.
3. Write a clear About page explaining who operates the site and what it offers.
4. Add a working Contact page or contact method.
5. Add a Privacy Policy that accurately explains the data and advertising technologies the site uses.
6. Add Terms of Use and clear game instructions where useful.
7. Ensure navigation works and pages are not empty.
8. Use original games/assets or assets you are legally allowed to publish.
9. Do not buy fake traffic, use bots, or ask people to click ads.
10. Never place ads in a way that encourages accidental clicks or blocks gameplay.
11. Read the current AdSense policies and application instructions.
12. Apply through the official AdSense account only when the site is ready.

Official AdSense approval help: https://support.google.com/adsense/answer/81904?hl=en

COPY-PASTE PROMPT

PROMPT 7 — SITE QUALITY AND ADSENSE READINESS REVIEW

Review my deployed mini-game website for content quality, usability, accessibility, privacy, and advertising readiness.

- Do not claim that approval is guaranteed.
- Identify thin or duplicated content and recommend genuinely useful original content.
- Check About, Contact, Privacy Policy, Terms, navigation, game instructions, mobile layout, loading speed, broken links, and console errors.
- Do not invent an address, company registration, testimonials, user numbers, or business facts.
- Do not create a fake privacy policy that claims technologies we do not use.
- Ask me for missing real-world details needed to complete legal/policy pages rather than making them up.
- Do not insert real ad code or ad placements until I provide the appropriate account/code and confirm where it should go.
- Report issues and fix the technical issues that can be safely fixed.
- Explain what still requires my review before I apply.

AdSense approval depends on Google's review and current policies. A privacy page alone does not guarantee approval.

Part 11 — Troubleshooting

PROBLEM: Vercel build fails

- Open the failed deployment's build logs.
- Copy the first meaningful error into Antigravity.
- Ask it to reproduce and fix the build locally.
- Run npm run build again if this is a Vite/React project.
- Commit and push the fix, or redeploy with the CLI.

> **Prompt:** "The Vercel deployment failed. Read the exact build log below, identify the root cause, reproduce it locally if possible, make the smallest correct fix, run the production build, and explain which files changed. Do not hide the error or claim success without a successful build."

PROBLEM: Website opens but game does not work

- Check the browser console.
- Check missing asset paths and filename capitalization.
- Check that the game initializes after the page loads.
- Check whether the game expects a backend or local file that is not deployed.
- Test the live deployment, not just localhost.

PROBLEM: Homepage works but refreshing a game route shows 404

- This may be a client-side routing configuration issue.
- Ask Antigravity to inspect the actual router and Vercel configuration, then add only the appropriate rewrite if needed.
- Do not copy a random vercel.json configuration from another framework.

PROBLEM: Images/audio work locally but not online

- Make sure assets are included in the repository.
- Use correct relative/public asset paths.
- Check case-sensitive file names.
- Do not reference C: paths or local-only files.

PROBLEM: Website is live but AdSense is not approved

- Read the reason in the official AdSense account.
- Improve original content, site completeness, navigation, and policy compliance.
- Do not repeatedly reapply without addressing the stated issues.
- No guide can guarantee approval.

PROBLEM: The site receives little traffic

- Hosting only makes the site available online.
- Publish useful original games and pages, improve performance and discoverability, share it legitimately, and learn from real user behavior.
- Never buy fake visits or encourage invalid ad clicks.

Part 12 — Final Checklist

Game

- ■ Game is actually playable.
- ■ Start/restart/pause and scoring work.
- ■ Touch and mouse controls work.
- ■ Mobile layout is usable.
- ■ No major console errors.

Website

- ■ Homepage and game links work.
- ■ Only completed games are listed as playable.
- ■ About/Contact/Privacy/Terms are accurate and accessible.
- ■ Assets load on the live domain.
- ■ No fake traffic, fake reviews, or fabricated earnings claims.

Vercel

- ■ Correct repository and project root selected.
- ■ Production build succeeds.
- ■ Public .vercel.app URL opens.
- ■ Game works on the deployed URL.
- ■ Refresh and navigation work.
- ■ Original source project remains safe.

AdSense

- ■ Site is complete and provides original value.
- ■ Current policies have been reviewed.

- No incentivized or invalid ad clicks.
- No claim of guaranteed approval or income.
- Application submitted only when ready.

— End of Guide —